

Clasificación automática de noticias de Página 12

Trabajo Práctico 1 — Web Mining

Víctor Ruiz

Maestría en Ciencia de Datos — Universidad Austral

May 29, 2026

Resumen

Se construyó un *pipeline* reproducible de extremo a extremo para clasificar notas del diario Página 12 en cuatro secciones (**Economía**, **El País**, **Sociedad** y **El Mundo**). El corpus se obtuvo mediante *crawling* de la versión online del diario (5999 artículos, mayo 2023–mayo 2026), se preprocesó con un pipeline NLP en español y se evaluaron representaciones clásicas (Bag-of-Words y TF-IDF) frente a una alternativa moderna (*sentence embeddings* multilingües), además de ensambles. El mejor modelo (TF-IDF + LinearSVC) alcanza un Macro F1 de 0.903 ± 0.011 en validación cruzada estratificada de 5 *folds* y 0.853 ± 0.004 en *validación temporal* con tres cortes. La caída moderada y *estable* entre los tres cortes indica que el modelo generaliza a noticias futuras y no está sobreajustado a un período particular.

1 Objetivo

El objetivo es entrenar un clasificador capaz de predecir a qué sección pertenece una noticia de Página 12, maximizando los resultados tanto de validación cruzada como de validación temporal, y demostrando que la solución generaliza (es decir, que los buenos resultados no se deben a *overfitting* ni a sesgo temporal del corpus).

2 Obtención de los datos (crawling)

2.1 El problema de la paginación

La consigna sugiere modificar el *crawler* de ejemplo `scrap_pagina12.py` y recorrer las páginas de cada sección mediante URLs del tipo `/secciones/economia?page=N`. Al implementarlo se detectó que **el parámetro `?page=N` es ignorado por el servidor**: todas las páginas devuelven el mismo bloque de ~ 23 noticias, porque la paginación se resuelve en el cliente con JavaScript (el sitio usa el CMS *Arc Publishing*). Un *crawler* de HTML estático, por lo tanto, no puede avanzar más allá de la primera pantalla de cada sección.

2.2 Solución: la API JSON de Arc Publishing

Inspeccionando el objeto `Fusion.globalContentConfig` embebido en el HTML se descubrió la API interna que alimenta la paginación:

```
GET /pf/api/v3/content/fetch/p12-section
?query={"page":N,"size":15,"primarySection":"/economia",...}
```

Esta API expone **paginación real** (~ 666 páginas por sección, ~ 10.000 notas cada una) y devuelve el cuerpo completo del artículo ya estructurado en `content_elements`. Sus ventajas sobre el *scraping* de HTML son: texto limpio directamente del CMS, sin selectores CSS frágiles, y respuestas JSON livianas (mucho más rápidas que descargar ~ 300 KB de HTML por nota). La consigna habilita explícitamente “cualquier herramienta, recurso y técnica”, de modo que esta decisión respeta

el espíritu del TP y mejora la calidad del dato. Se conservó además `scrap_pagina12.py` con las correcciones de *regex* y *headers* como referencia.

2.3 Control del sesgo temporal

La consigna advierte que descargar noticias de un período corto introduce sesgo: el modelo aprendería los hechos coyunturales de ese período y no generalizaría. Para evitarlo, en lugar de tomar las primeras K páginas (las más recientes), el *crawler* **distribuye uniformemente las páginas a lo largo de toda la historia** de cada sección ($step = \lfloor p_{\max}/p_{\text{necesarias}} \rfloor$). El resultado es un corpus que cubre de mayo 2023 a mayo 2026, con representación de muchos meses distintos (Figura 1). Se añadió *rate limiting* con *back-off* ante respuestas 429 y *jitter* aleatorio para no saturar el servidor.

2.4 Corpus resultante

Sección	Artículos
Economía	1500
El País	1500
Sociedad	1500
El Mundo	1499
Total	5999

Tabla 1: Distribución del corpus por sección (~ 1.500 notas por clase, prácticamente balanceado).

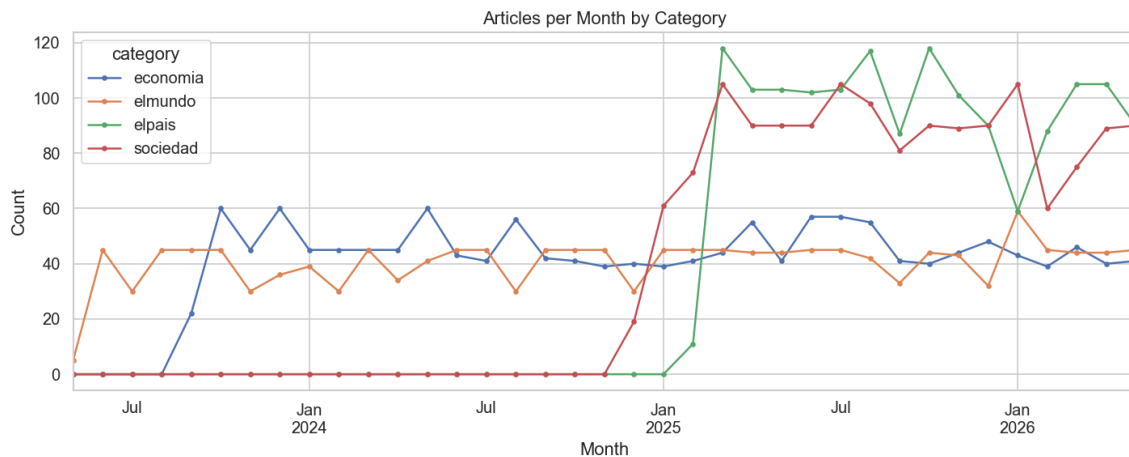


Figura 1: Artículos por mes y sección. La cobertura temporal amplia mitiga el sesgo de coyuntura.

3 Extracción y preprocesamiento de texto

De cada artículo se extrajeron *título*, *bajada*, *cuerpo*, *fecha*, *autor* y *URL*. Las notas con cuerpo menor a 150 caracteres se descartan (zócalos, galerías, vivos). El texto se procesó con un pipeline NLP configurable en español:

- lectura en **UTF-8** (la consigna advierte sobre el *encoding*: una lectura incorrecta rompe los acentos);
- eliminación de etiquetas HTML residuales y normalización Unicode (NFC);

- pasaje a minúsculas y eliminación de puntuación/números;
- remoción de *stopwords* en español (lista de la cátedra);
- *stemming* Snowball (opcional, evaluado).

La mediana resultante es de ~ 389 *tokens* por artículo (Figura 2), suficientemente larga para una clasificación léxica robusta.

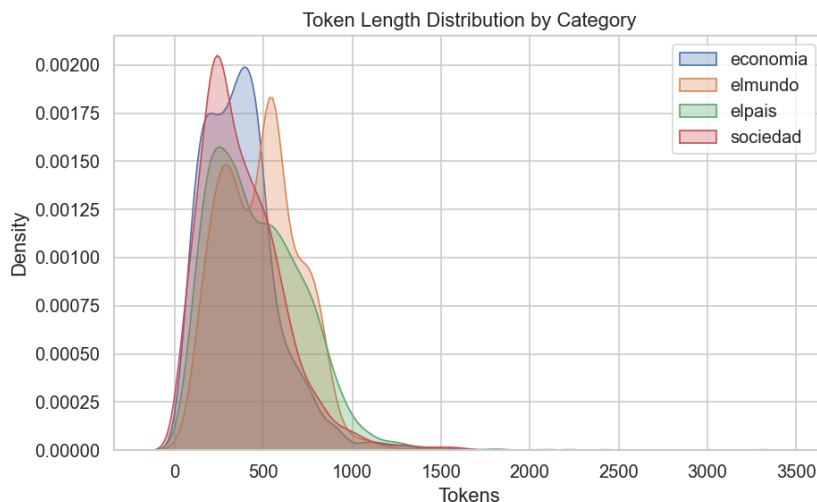


Figura 2: Distribución de longitud (tokens) por sección.

4 Representaciones y modelos

Se evaluaron, como pide la consigna, un **baseline con Bag-of-Words/TF-IDF** y al menos **una alternativa moderna** de representación (*sentence embeddings*). Todos los modelos se encapsularon en Pipeline de **scikit-learn** para evitar fuga de información entre *folds* (el vectorizador se ajusta solo con el *train* de cada *fold*).

BoW + Naive Bayes CountVectorizer (unigramas) + Multinomial NB. Baseline explícito, rápido e interpretable.

BoW + Regresión Logística CountVectorizer (1-2 gramas) + LR.

TF-IDF + Regresión Logística ponderación TF-IDF *sublinear* (1-2 gramas) + LR.

TF-IDF + LinearSVC igual representación, clasificador de margen máximo lineal. *Mejor modelo*.

Sentence embeddings + LR paraphrase-multilingual-MiniLM-L12-v2 (alternativa moderna). Se embebe *título + bajada + inicio del cuerpo* para respetar el límite de 128 *tokens* del modelo.

Ensambls *soft voting* y *stacking* (meta-learner LR) sobre los modelos lineales.

5 Evaluación

5.1 Validación cruzada (5-fold estratificado)

Se reporta *accuracy* y **Macro F1** (esta última trata a las cuatro clases por igual, lo importante con clases casi balanceadas pero donde interesa el desempeño por sección). La Tabla 2 resume los resultados; la Figura 3 los compara visualmente.

Modelo	Accuracy	Macro F1
TF-IDF + LinearSVC	0.904 $\pm$ 0.011	0.903 $\pm$ 0.011
Ensamble (stacking)	0.901 $\pm$ 0.009	0.901 $\pm$ 0.009
TF-IDF + Reg. Logística	0.900 $\pm$ 0.010	0.900 $\pm$ 0.010
Ensamble (soft voting)	0.890 $\pm$ 0.009	0.890 $\pm$ 0.009
BoW + Reg. Logística	0.885 $\pm$ 0.007	0.885 $\pm$ 0.007
BoW + Naive Bayes	0.883 $\pm$ 0.006	0.882 $\pm$ 0.006
Sentence embeddings + LR	0.818 $\pm$ 0.005	0.818 $\pm$ 0.005

Tabla 2: Validación cruzada 5-fold estratificada. Media $\pm$ desvío entre folds.

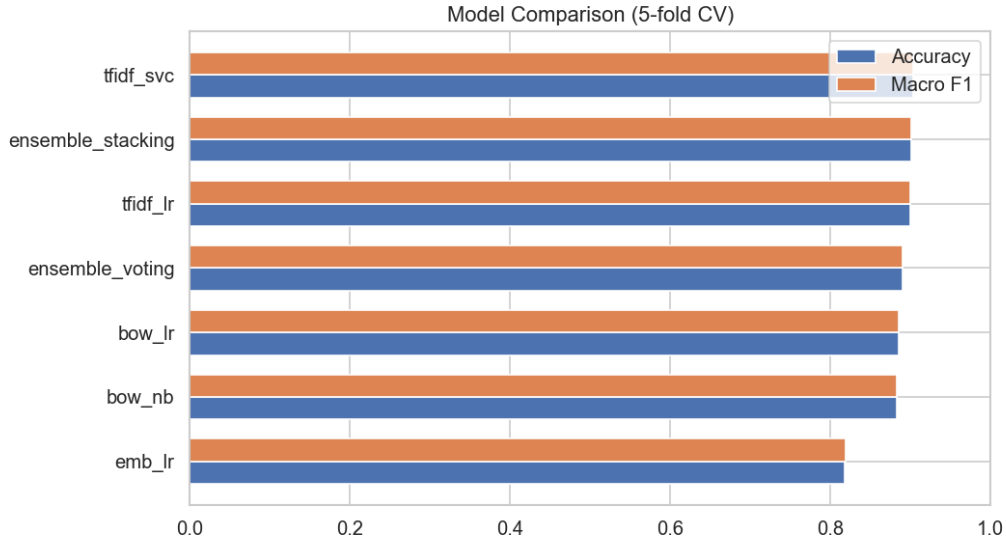


Figura 3: Comparación de modelos (Macro F1 y Accuracy en CV).

5.2 Matriz de confusión y clases difíciles

La Figura 4 muestra la matriz de confusión promediada en CV del mejor modelo (*recall* por clase en la diagonal). Respondiendo a las preguntas concretas de la consigna:

- **¿Hay una clase más difícil?** Sí. **El País** es claramente la sección más difícil (*recall* $\approx 85\%$), seguida de *Sociedad* ($\approx 87\%$). En el otro extremo, *El Mundo* ($\approx 95\%$) y *Economía* ($\approx 94\%$) son las mejor separadas, gracias a su vocabulario característico (geopolítico: “ONU”, “Gaza”, “Ucrania”; y económico: “dólar”, “inflación”, “BCRA”).
- **¿Qué pares se confunden más?** El par más confundido es **El País** $\leftrightarrow$ **Sociedad** ($\approx 7\%$ en ambos sentidos, prácticamente *simétrico*): ambas secciones tratan temas nacionales (política, salud, educación, derechos) con vocabulario solapado.
- **¿Hay confusión asimétrica?** Sí. El par **El País** $\rightarrow$ **Economía** ($\approx 5,4\%$) supera al inverso **Economía** $\rightarrow$ **El País** ($\approx 3,6\%$): una nota de política económica (“el Gobierno anunció medidas para contener la inflación”) tiene señales léxicas de ambas, pero el sesgo de error apunta *de El País hacia Economía*, no al revés. Es decir, las notas políticas con jerga económica tienden a ser etiquetadas como Economía más que las notas económicas como política.

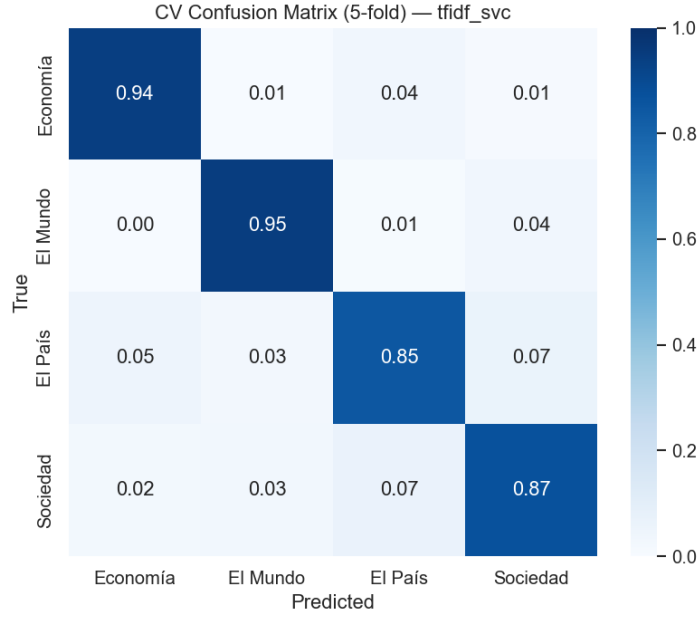


Figura 4: Matriz de confusión (CV 5-fold, normalizada por fila) del mejor modelo.

5.3 Validación temporal (múltiples cortes)

Se entrena con las notas anteriores a una fecha de corte y se evalúa con las posteriores, simulando el uso real del modelo sobre noticias futuras. Como pide la consigna (“idealmente, más de un corte temporal”), se evaluaron **tres cortes** — los últimos 3, 6 y 9 meses como conjunto de test — para descartar que un buen resultado sea un accidente de un período puntual.

Corte	Fecha	Train	Test	Accuracy	Macro F1
3m	2026-02-28	5185	814	0.853	0.858
6m	2025-11-29	4403	1596	0.848	0.853
9m	2025-08-29	3570	2429	0.846	0.849
		Promedio		0.849 ± 0.003	0.853 ± 0.004

Tabla 3: Validación temporal en tres horizontes. El test es siempre posterior al train. La estabilidad entre cortes confirma que el resultado no es casual.

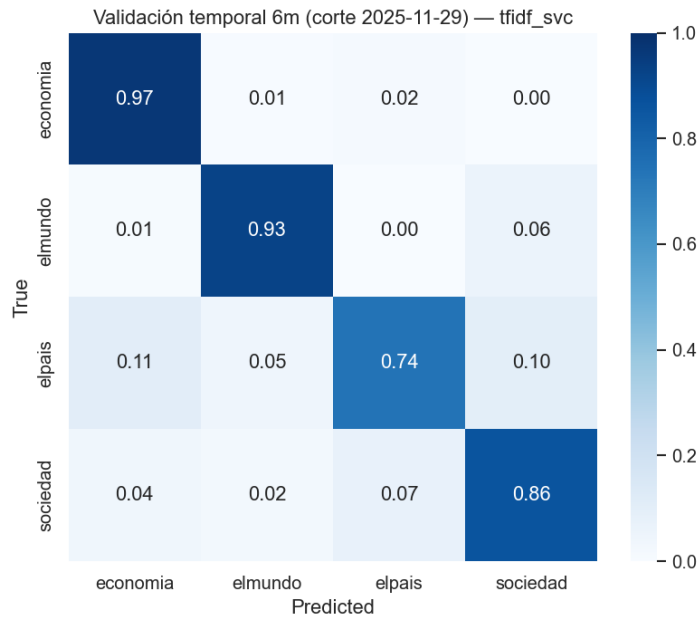


Figura 5: Matriz de confusión en validación temporal (corte de 6 meses).

5.4 ¿Hay diferencia entre CV y validación temporal?

La diferencia es **moderada pero muy estable**: el Macro F1 cae de 0.903 ± 0.011 (CV) a 0.853 ± 0.004 (temporal), unos 5 puntos. Lo relevante es que los tres cortes dan valores casi idénticos (0,858 / 0,853 / 0,849) pese a que el test crece de 814 a 2.429 noticias: el resultado *no es un accidente* de un período puntual. La explicación de la pequeña caída es doble: (i) en CV el modelo puede ver vocabulario contemporáneo al test (mezcla de fechas), mientras que en temporal entrena solo con el pasado; (ii) aun así el vocabulario que define cada sección (jerga económica, organismos internacionales, actores políticos, temas sociales) es **estable en el tiempo**, por lo que un modelo entrenado con notas viejas sigue clasificando bien notas futuras. Una caída *grande* habría delatado sesgo temporal o *overfitting* a la coyuntura del *train*; una caída chica y estable confirma que la solución generaliza.

6 Discusión de decisiones de diseño

Por qué TF-IDF supera a los embeddings. La tarea es de *dominio léxico*, no de semántica profunda: las secciones se distinguen por vocabulario específico (Economía: “dólar”, “inflación”, “BCRA”; El Mundo: “ONU”, “Gaza”, “Ucrania”). TF-IDF captura exactamente esas señales discriminativas. Los *embeddings* de MiniLM comprimen el texto en un espacio semántico continuo donde términos de distinta sección quedan próximos (p.ej. “inflación” y “pobreza”), perdiendo poder discriminativo; además su límite de 128 *tokens* obliga a truncar artículos cuya mediana es de ~ 389 *tokens*. Por eso el *baseline* clásico, bien ajustado, resulta más fuerte que la alternativa moderna en esta tarea concreta.

Por qué el ensamble no mejora al mejor modelo individual. El *stacking* reduce la varianza entre *folds* pero no mejora la media: los modelos base cometen **errores correlacionados** (todos confunden El País con Sociedad y Economía), de modo que el meta-learner no encuentra señal complementaria. El ensamble agrega valor cuando los errores son independientes, lo que aquí no ocurre.

Generalización (no overfitting). Tres salvaguardas sostienen que el resultado generaliza: (i) toda la vectorización vive dentro del **Pipeline** y se ajusta solo con el *train* de cada *fold*, evitando fuga de información; (ii) la cobertura temporal amplia del corpus evita el sesgo de coyuntura; y (iii) la validación temporal con múltiples cortes confirma desempeño estable sobre noticias estrictamente futuras.

7 Conclusiones

Se obtuvo un clasificador de cuatro secciones con Macro F1 de 0.903 ± 0.011 (CV) y ~ 0.853 (temporal), usando una representación TF-IDF de n -gramas con un clasificador lineal de margen máximo. La alternativa moderna (*embeddings*) fue evaluada y resultó inferior por la naturaleza léxica de la tarea, lo que constituye un hallazgo en sí mismo. La principal fuente de error es la ambigüedad real entre *El País* y *Economía*, no una deficiencia del modelo. La pequeña diferencia entre validación cruzada y temporal demuestra que la solución generaliza a noticias futuras.

A Reproducibilidad

```
python scripts/run_scraper.py --articles-per-class 1500 # crawling (Arc API)
python scripts/parse_html.py                          # preprocesamiento NLP
python scripts/train.py                                # entrenamiento + CV + temporal
python scripts/generate_report.py                      # reporte
python scripts/download_sample_html.py --per-class 12  # muestra HTML por sección
```

Repositorio: https://github.com/vic-ruiz/web_mining